Exercise 12

Elisa Schöpf, Leon Krieger, Nikola Adzic

June 13, 2026

Contents

1 Setup and Basic Execution **1**

2 Profiling **1**

3 Code Understanding **2**

4 Optimization **3**

1 Setup and Basic Execution

LCC3 fib.lua results:

```
100 x fibonacci_naive(30)     time:  13.7866 s  --  832040
10000000 x fibonacci_tail(30) time:  15.8393 s  --  832040
25000000 x fibonacci_iter(30) time:  12.2847 s  --  832040
```

The median of averages from 20 runs was 13.7609 s

2 Profiling

Initially we used `profile.lua`(`https://github.com/2dengine/profile.lua`) to profile the benchmark. However this only profiles the Lua script itself, showing which Lua functions take the most time. It confirmed that the Fibonacci functions are the main workload. Since we are not allowed to modify the Lua script, knowing which script-level function is slow does not give us any starting point for optimizing the interpreter.

We then used `perf` to profile the program. This result was more useful because it breaks down the time spent in the C source code.

```
[...]
# Samples: 116K of event 'cycles:u'
# Event count (approx.): 114068069768
#
# Children      Self  Command  Shared Object     Symbol
# ........  ........  .......  .................  ......................
#
    79.12%    78.18%  lua      lua                [.] luaV_execute
            |
            |--14.88%--0
            |          |
            |           --14.61%--luaV_execute
            |
            |--7.37%--0x281c900060080
            |          luaV_execute
[...]
```

The programs spends ∼79% of its time in `luaV_execute`, which is the core VM loop, meaning we should focus our efforts there. It also shows some time is spent in `luaD_precall` and `luaD_pretailcall` (∼15% combined). This tells us that the overhead isn't just in executing instructions, but also in the management of the Lua stack and function calls.

We also manually added opcode statistics to get a closer look at what `luaV_execute` does internally, which tells us what the hottest parts of the code are and gives us hints where our optimizations could start.

```
--- Opcode Statistics ---
EQI             : 310000031
LTI             : 271946237
GETTABUP        : 135973068
ADD             : 135973068
MOVE            : 35000103
LOADI           : 25000001
CLOSURE         : 10000001
VARARGPREP      : 4
SETTABUP        : 3
---------------------------
```

3 Code Understanding

There are two major phases in the Lua interpreter, script compilation and script execution. The interpreter first goes through the program code to convert it into Lua bytecode instructions. This bytecode is then executed by Lua's virtual machine. The majority of it is handled by the `luaV_execute` function. This function consists of a large switch statement which checks against different OP_IDs to determine what to do.

The compile time was measured by using `luac`, which only compiles the program and doesn't execute it. This pre-compiled bytecode was then used to measure the execution time. We can clearly see that the compilation time for the benchmark is minuscule compared to the execution time.

```
Compilation Median:     00.0083 sec
Execution Median:       37.4024 sec
```

As seen on the `perf` report, the interpreter spends a lot of time in `luaV_execute`. As expected, this takes a lot of time and is inefficient.
`LUA_USE_JUMPTABLE` helps with that by enabling direct-threaded dispatch in the virtual machine using GCC/Clang computed gotos. This replaces the large switch statement, thus increasing performance. The jumptable only works with gcc/clang and is turned automatically on when compiled with either of those. It can be turned off by passing `-DLUA_USE_JUMPTABLE=0` to the compiler.

```
 Function                        :  With      :  Without   :  Speedup :
100 x fibonacci_naive(30)        :  13.7866 s :  13.8432 s :  x 1.004 :
10000000 x fibonacci_tail(30)    :  15.8393 s :  19.2325 s :  x 1.214 :
25000000 x fibonacci_iter(30)    :  12.2847 s :  14.7320 s :  x 1.199 :
Median of averages               :  13.7609 s :  14.7412 s :  x 1.071 :
```

The speedup column reveals how much faster the program is executed when gotos are utilized. The performance gains do differ across different implementations. The recursive implementation `fibonacci_naive` shows barley any improvement, most likely due the overhead of the function constantly calling itself and managing those recursive layers. Both the tail-recursive and iterative implementations see a speedup of ∼20%. They spend most of their time executing the same small sequence of operations over and over again, which the jumptable improves.

4 Optimization

To measure performance, at least 5 benchmarks were run. Then the median of the average from all 3 values of the benchmark were compared. This way, an optimization that only benefits once variation but slows down the others would not show up as more performant. The following optimizations were implemented:

Applying heuristics to OP_ADD resulted in 1.4% performance increase. Since the program only uses ints, we used a faster calculation for these cases:

```
// Apply heuristic of ints
if (ttisinteger(v1) && ttisinteger(v2)) {
  lua_Integer i1 = ivalue(v1);
  lua_Integer i2 = ivalue(v2);
  setivalue(s2v(RA(i)), intop(+, i1, i2));
  pc++; /* skip following MMBIN */
  vmbreak;
}
```

Using the heuristic optimizations above and `-fprofile-generate` + `-fprofile-use` resulted in a 9.6% performance increase.